



Faculty of Engineering - Ain Shams University (ASUENG)
Department of Computer and Systems Engineering (CSE)

Phase 1 (System Design)

Course code: CSE352s

Course Title: Parallel and Distributed Systems

Project Name: Distributed Online Marketplace System

Table of Contents

Team Members	3
1. Introduction.....	4
2. Target Beneficiaries of the Project.....	4
3. Adopted Programming Language	4
4. System Architecture	5
6. Sequence Diagrams and User Flow	9
7. Time Plan	16
8. Selected technologies	16
9. Resources Needed.....	16

Table of Figures

Figure 1 High-Level Architecture	5
Figure 2 Distributed Database Design.....	8
Figure 3 login Sequence	9
Figure 4 login user flow diagram	9
Figure 5 Create Account Sequence	10
Figure 6 Create account user flow diagram.....	10
Figure 7 Search items sequence diagram	11
Figure 8 Search items user flow diagram	11
Figure 9 Add / edit / remove item	12
Figure 10 Add / edit / remove item user flow diagram.....	12
Figure 11 Purchase item flow sequence diagram.....	13
Figure 12 Purchase item user flow diagram	13
Figure 13 Deposit cash	14
Figure 14 Deposit cash user flow diagram	14
Figure 15 view Account info Sequence	15
Figure 16 View account user flow diagram.....	15

Team Members

Name	Code
Youssef Medhat Mahmoud	2200626
Omar Ahmed Mohamed Ali	2200267
Abdallah Hossam Ragab	2100844
Abdulrahman Ibrahim Mohamed	2200960
Ahmed Mohamed Mohyeldin	2200955
Ahmed Ramadan Yassin	2200372
Ibrahim Mahmoud Ibrahim	2200182
Mostafa Ahmed Abd El Sattar	2201092
Tarek Hossam Eldin	2200595
Abdelrahman Ashour Hassan	2101736
Mohamed Hamdy Youssif Ahmed	2200416
Ahmed Ayman Ismael	2200853

1. Introduction

This document outlines the design and implementation details of a distributed online marketplace system.

The system allows users to create accounts, manage their virtual wallets by depositing cash, and buy or sell items securely. To achieve a highly scalable and robust architecture, the system is designed as a group of distributed microservices communicating via low-level Java Sockets.

This approach ensures appropriate resource distribution and fulfills the requirement of building a decentralized platform where data is partitioned across multiple nodes.

2. Target Beneficiaries of the Project

The delivered software application serves multiple entities:

- **Regular Users (Buyers):** Can search for items by name or brand, deposit cash, and purchase goods seamlessly.
- **Sellers:** Can add, edit, or remove items for sale, specify prices, and manage their inventory.
- **External Store Owners:** Through specific interfaces, other stores can integrate to sell their products on our platform.
- **System Administrators:** Can generate reports regarding transactions performed across the distributed nodes.

3. Adopted Programming Language

The project is entirely implemented using **Java**.

We utilized low-level Java ServerSockets and Sockets for all inter-node communication. To satisfy the web services requirement, three core functionalities (Search, Login, and Reporting) were implemented using a custom HTTP-like parser over Sockets to mimic RESTful services from scratch.

4. System Architecture

Architecture Style

The system follows a **Distributed Microservices Architecture** where the application is divided into independent services that communicate through REST APIs.

High-Level Architecture

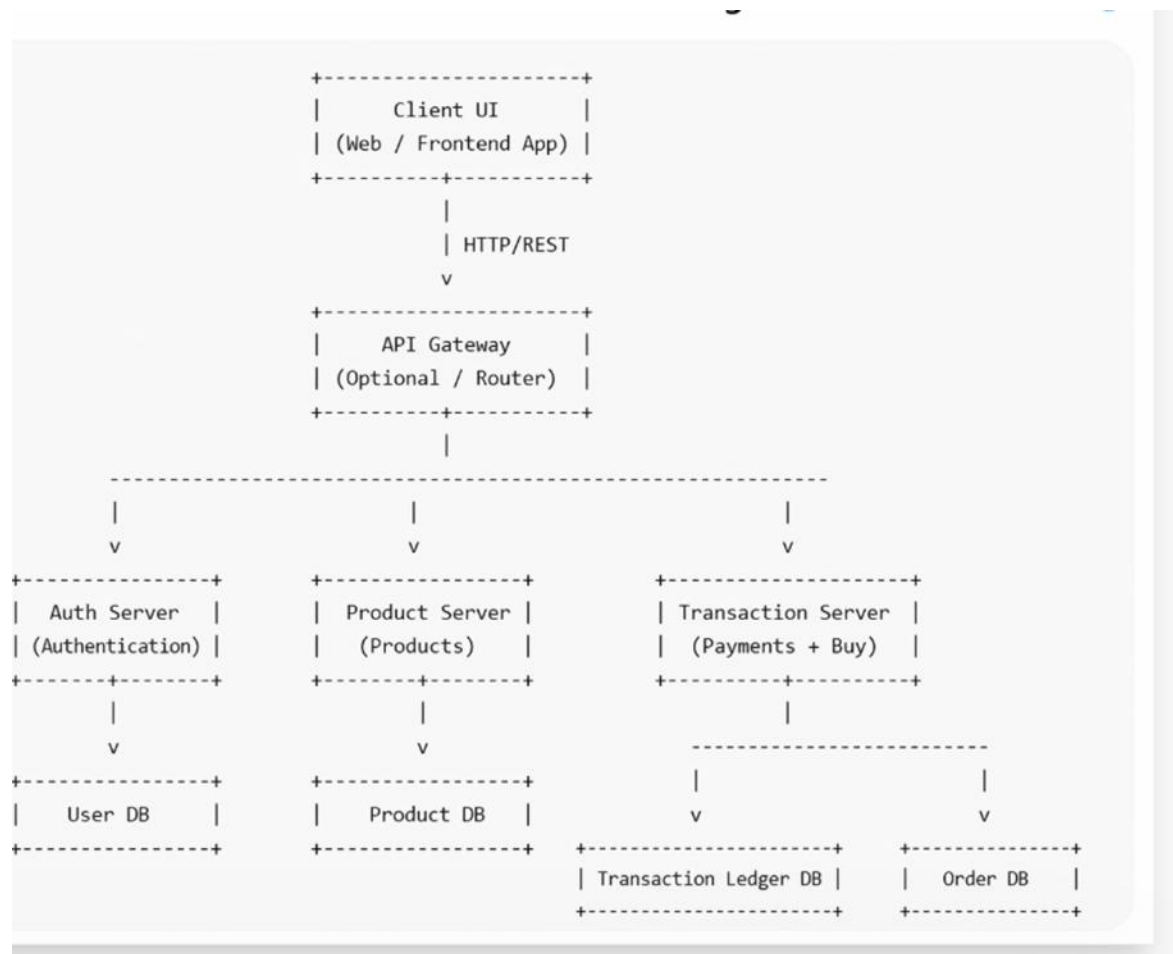


Figure 1 High-Level Architecture

1) Client Layer

- Represents the user interface (Web application or frontend).
- Sends HTTP requests to backend services.
- Displays results such as items, account info, and transactions.

2) API Layer Acts as a single entry point to the system.

- Routes requests to appropriate microservices.
- Helps in simplifying client communication.

3) Microservices Layer

Auth Service

Responsible for:

- User registration
- Login authentication
- Managing user accounts and balances

Item Service

Responsible for:

- Adding items for sale
- Editing and deleting items
- Searching items by name or brand

Transaction Service

Responsible for:

- Handling purchases
- Transferring money between users
- Recording transaction history

4) Distributed Database Layer

Each service has its own database (or schema):

- User DB → stores users and balances
- Item DB → stores products
- Transaction DB → stores purchases and logs
- Order DB → stores orders and order details

This ensures **data separation and service independence**

Communication Between Services

- Services communicate using **JavaSockets**
- Data exchange format: **JSON**

Examples

Transaction Service → calls User Service to check balance

Transaction Service → calls Item Service to check item availability

Distributed Model Justification

The system uses a **Microservices-based distributed model** because:

- Each service is independent and scalable
- Services can be developed and deployed separately
- Failure in one service does not affect others
- Easier maintenance and team collaboration

The database is fragmented and distributed across the microservices to prevent single points of failure.

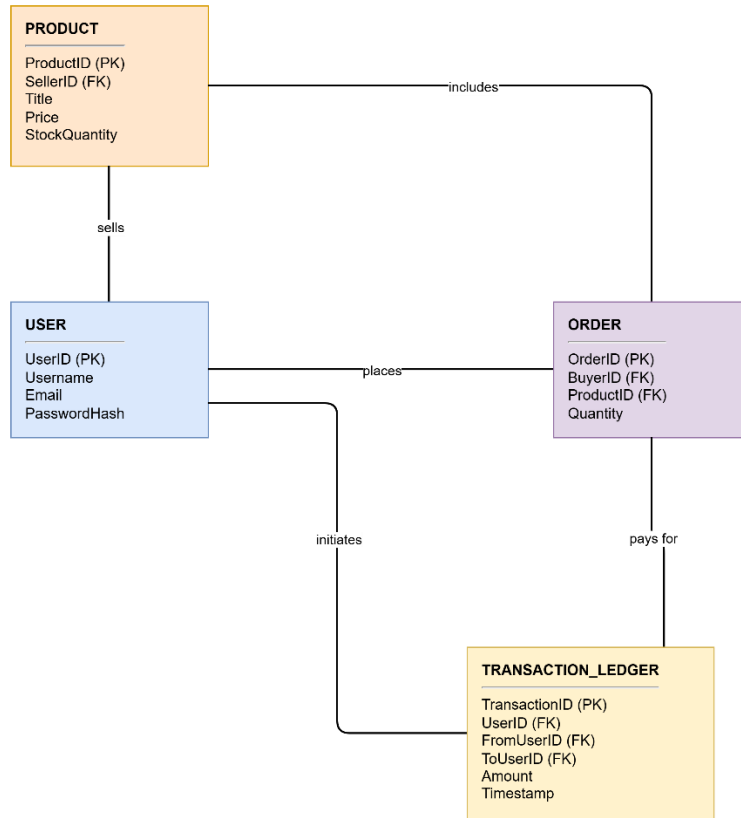


Figure 2 Distributed Database Design

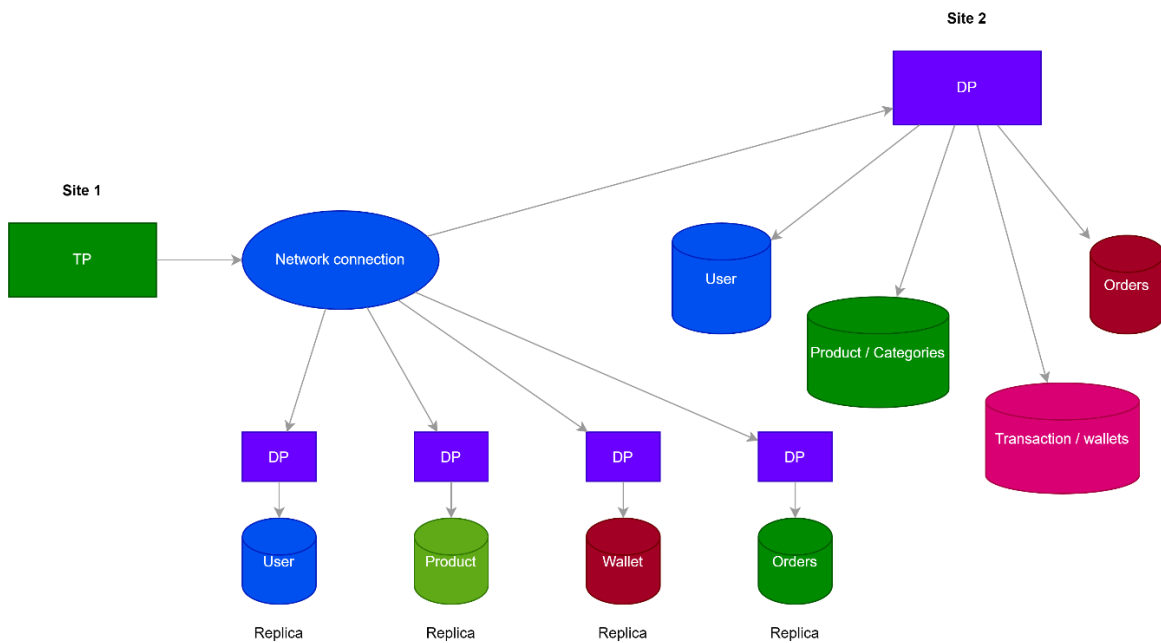


Figure 3 Distributed Database Model

6. Sequence Diagrams and User Flow

The sequence diagrams provided in this report serve as the technical reference for the system's internal mechanics.

In contrast, the user flows prioritize the behavioral logic of the platform using plain, non-technical language.

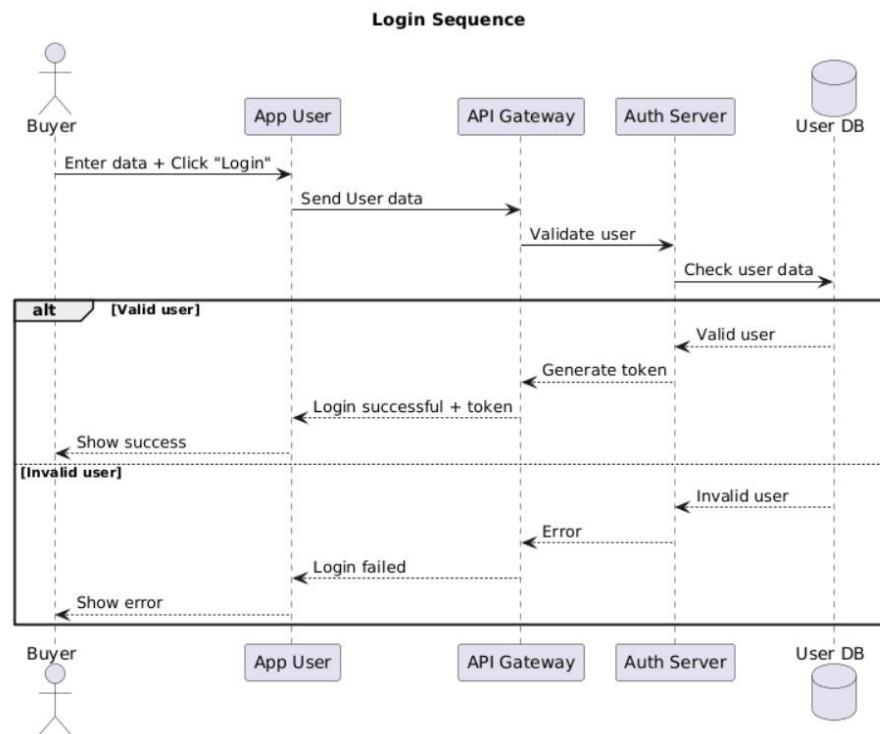


Figure 4 login Sequence

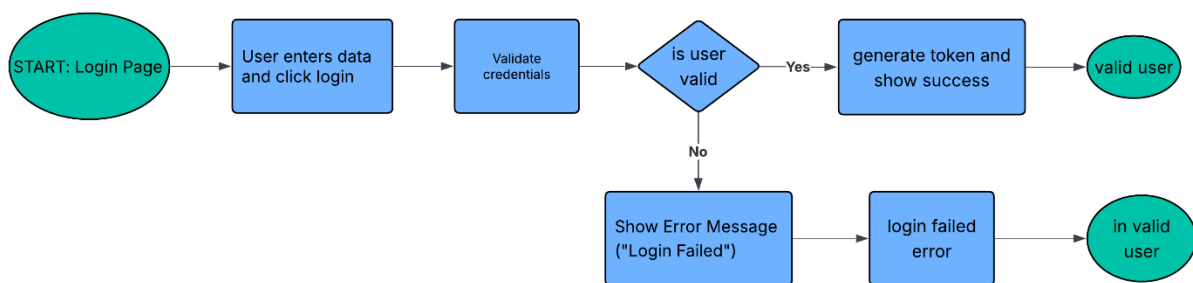


Figure 5 login user flow diagram

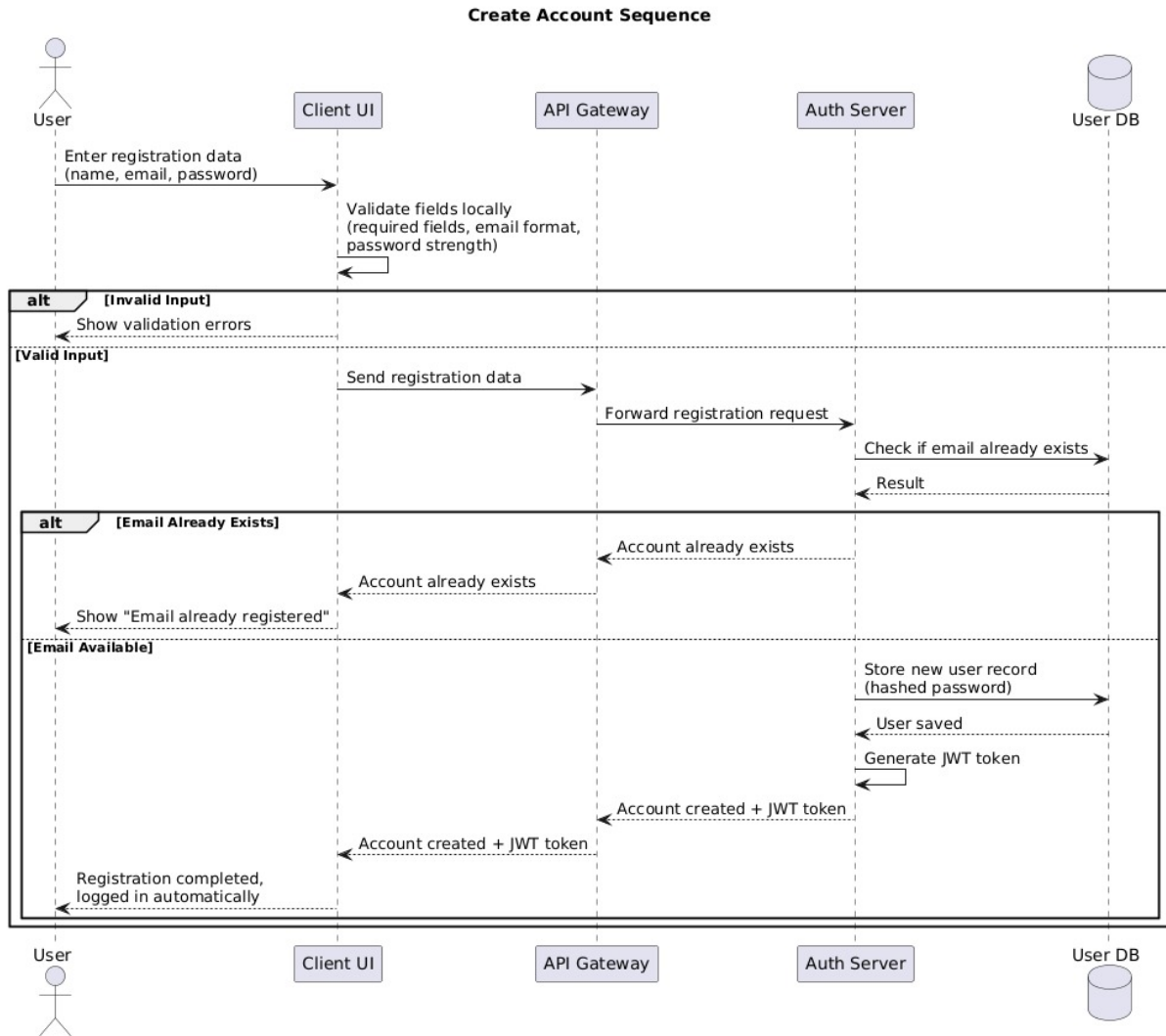


Figure 6 Create Account Sequence

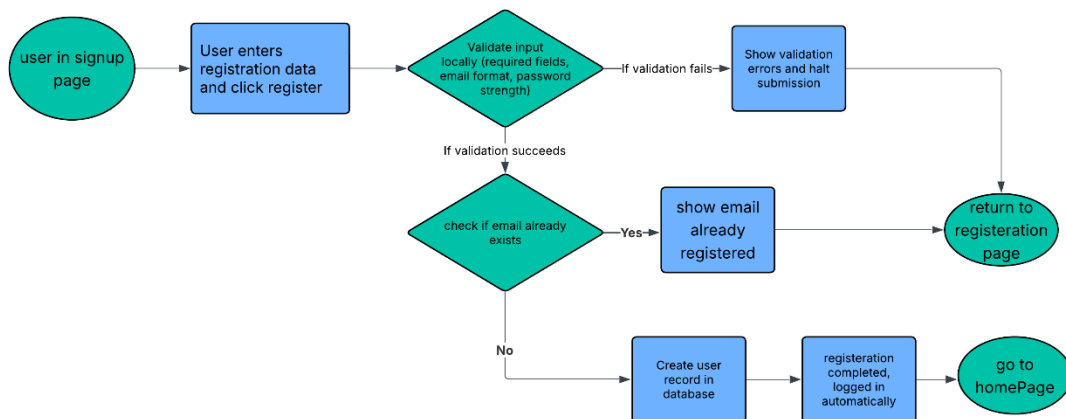


Figure 7 Create account user flow diagram

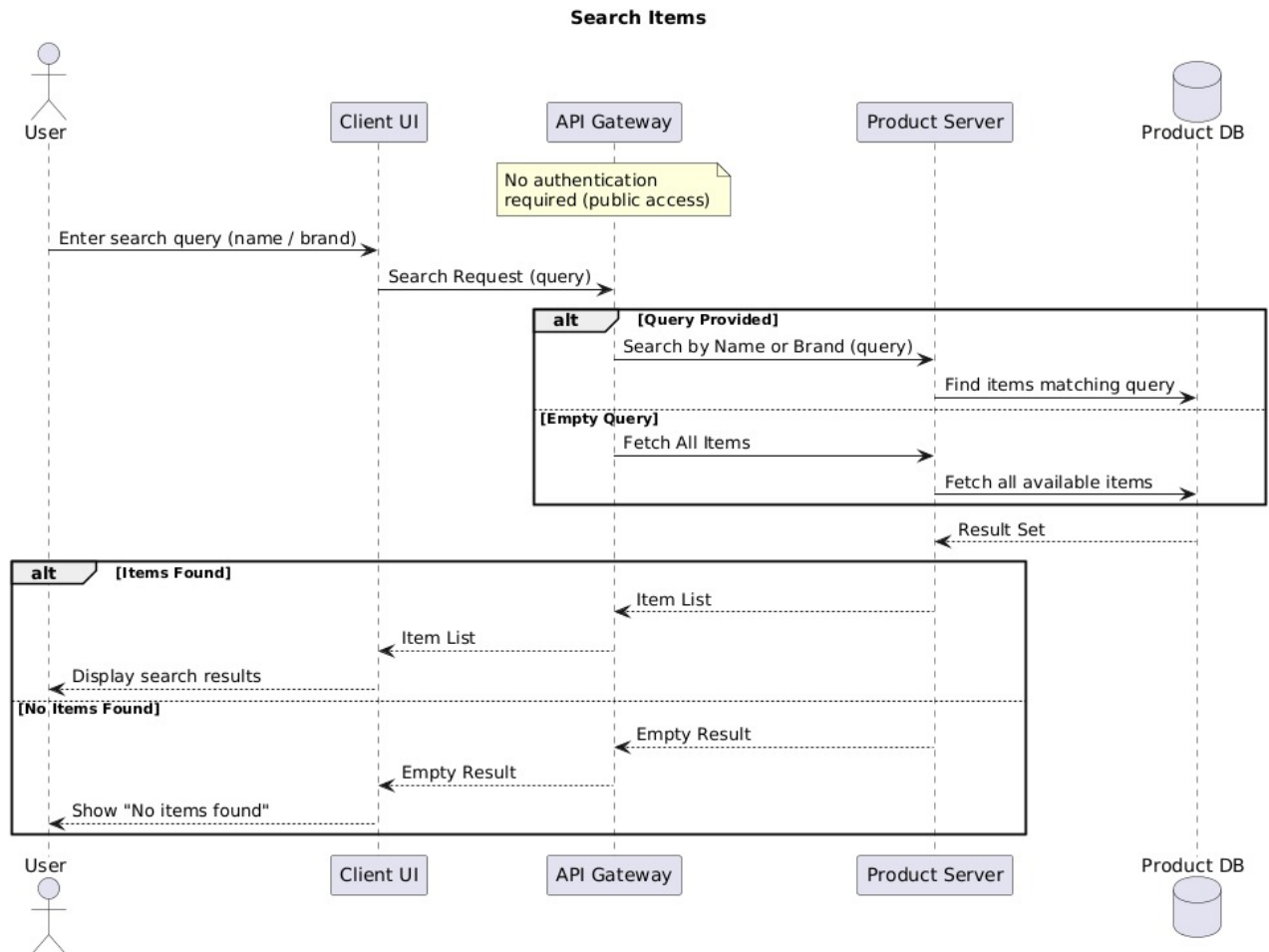


Figure 8 Search items sequence diagram

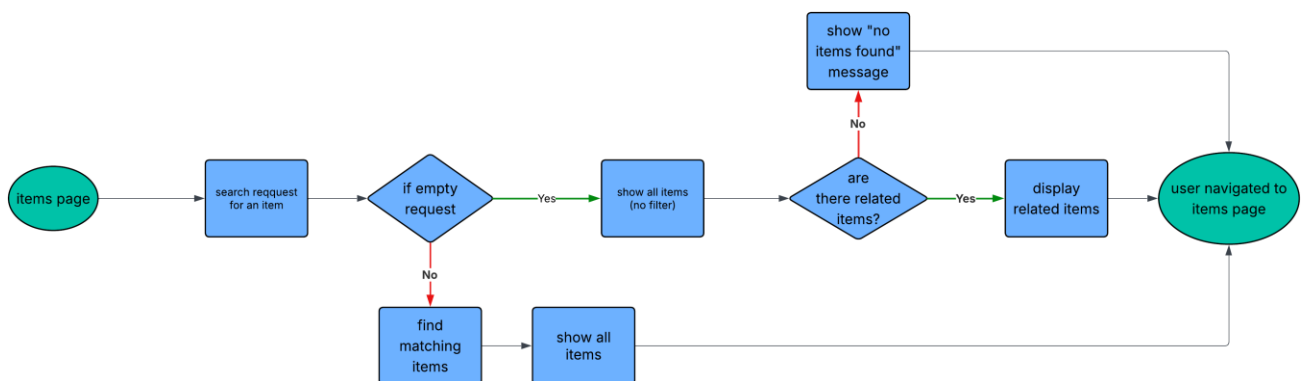


Figure 9 Search items user flow diagram

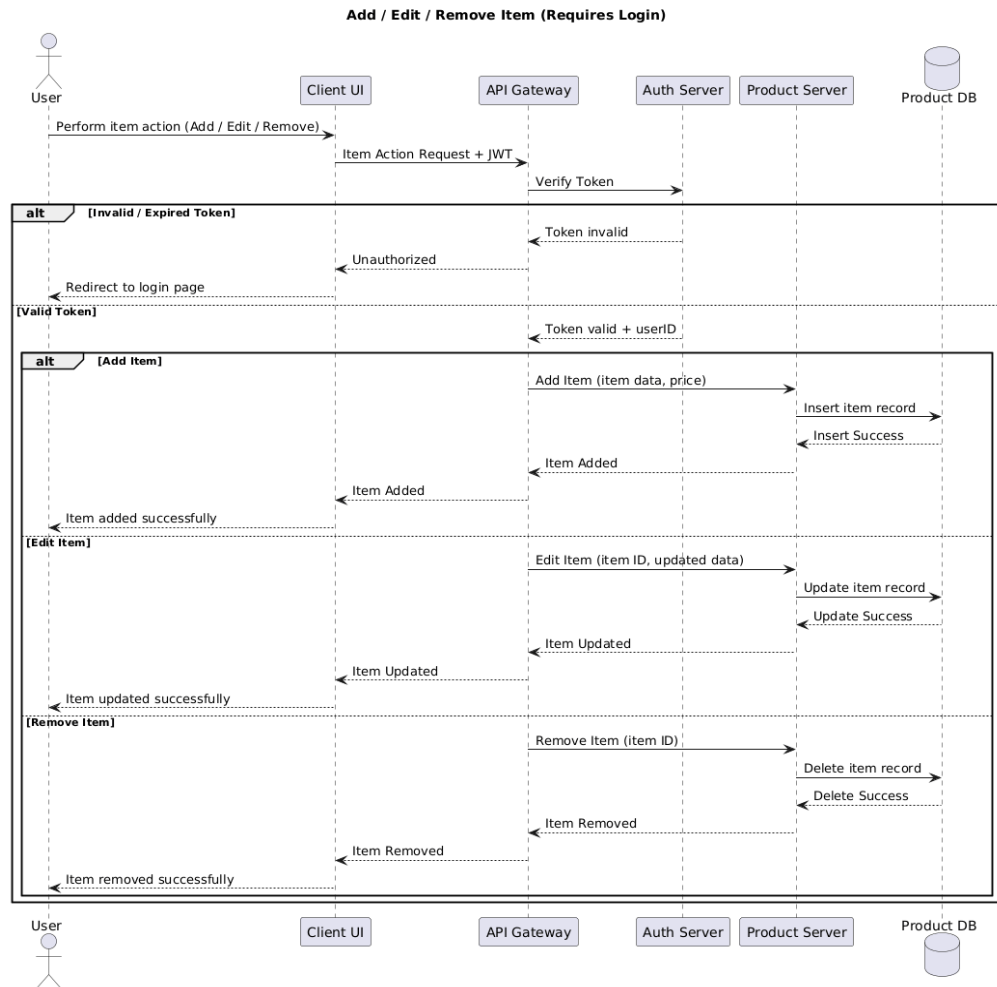


Figure 10 Add / edit / remove item

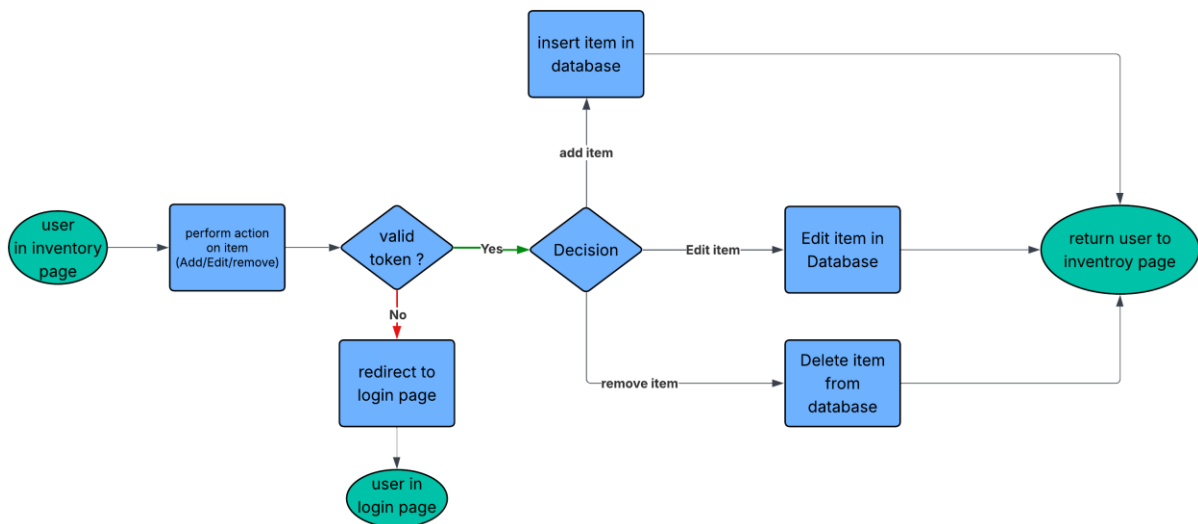


Figure 11 Add / edit / remove item user flow diagram

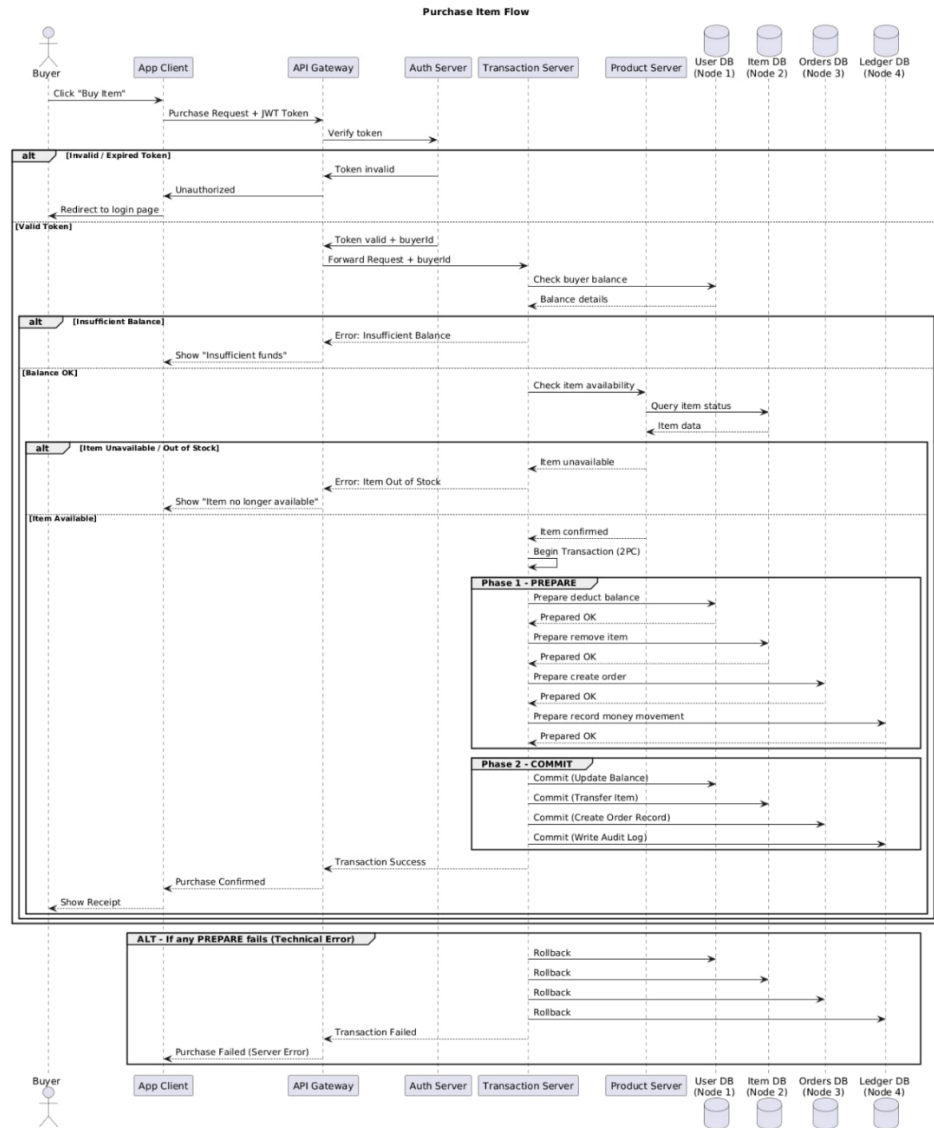


Figure 12 Purchase item flow sequence diagram

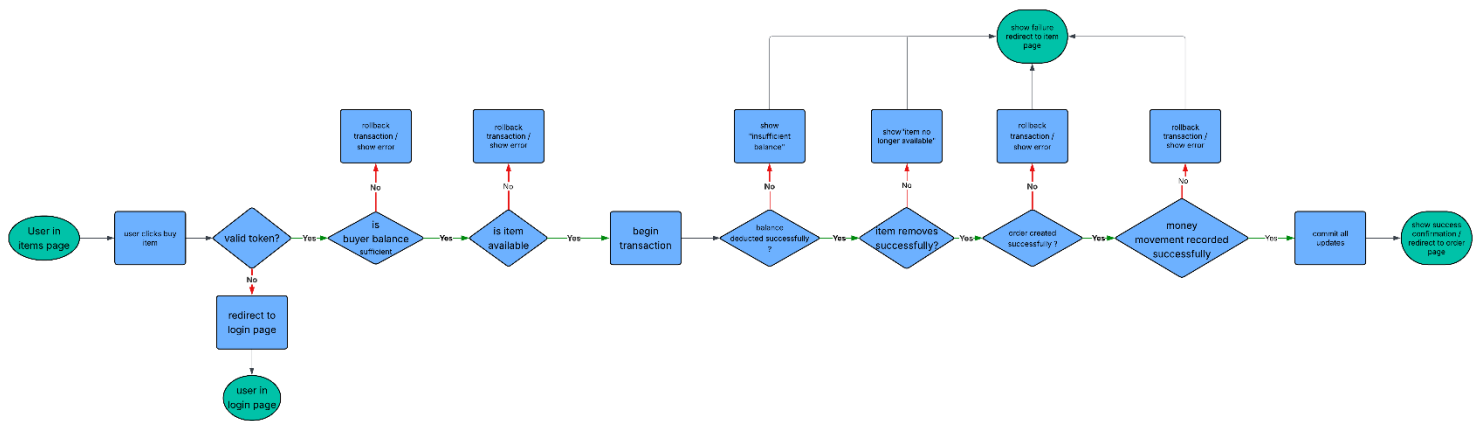


Figure 13 Purchase item user flow diagram

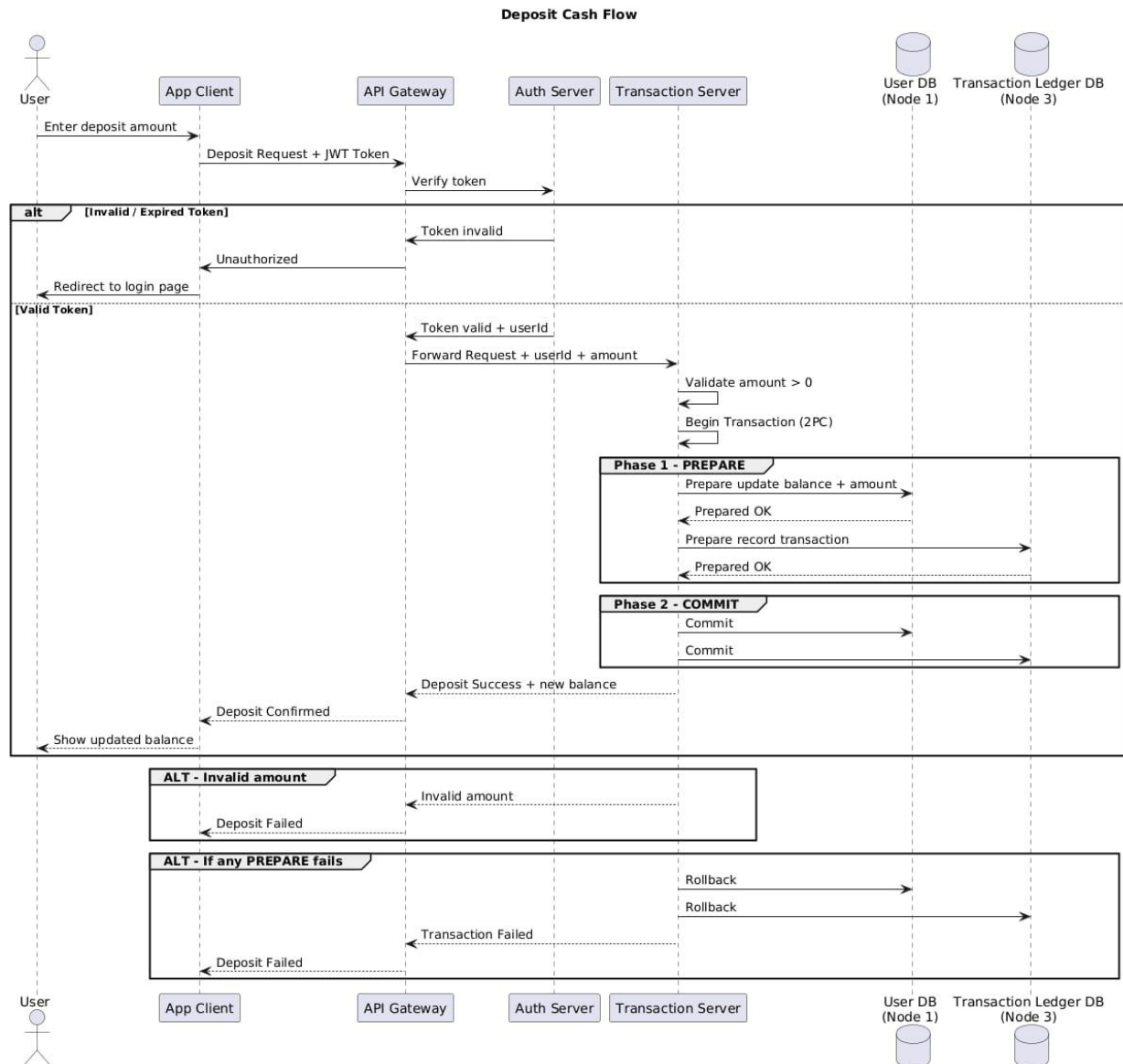


Figure 14 Deposit cash

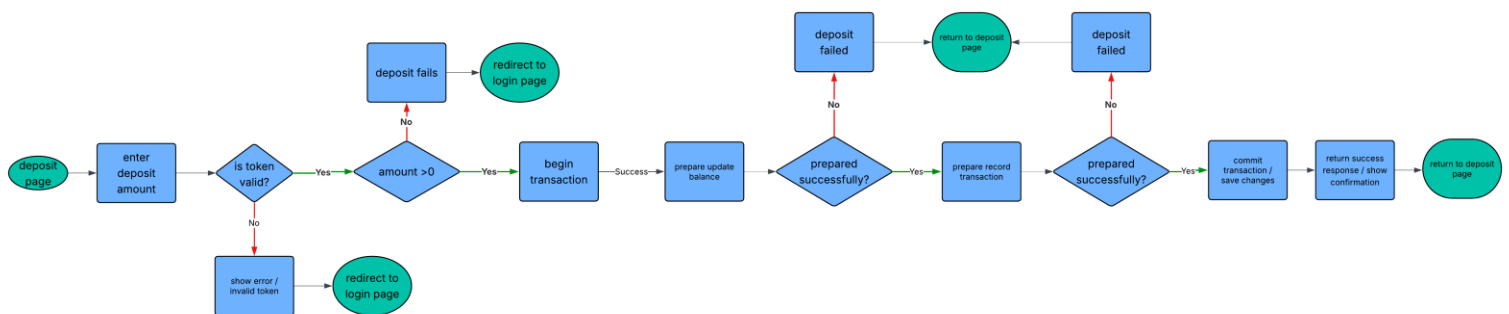


Figure 15 Deposit cash user flow diagram

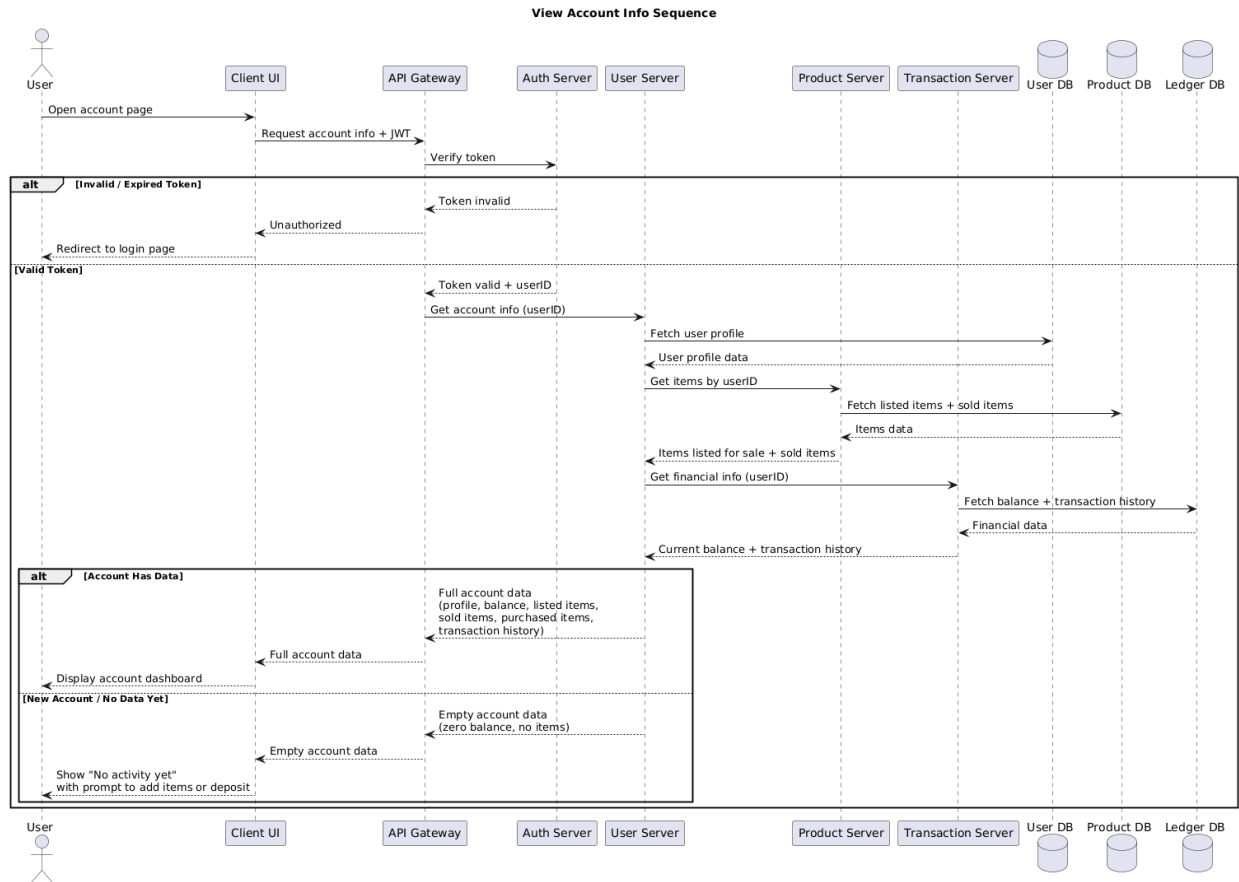


Figure 16 view Account info Sequence

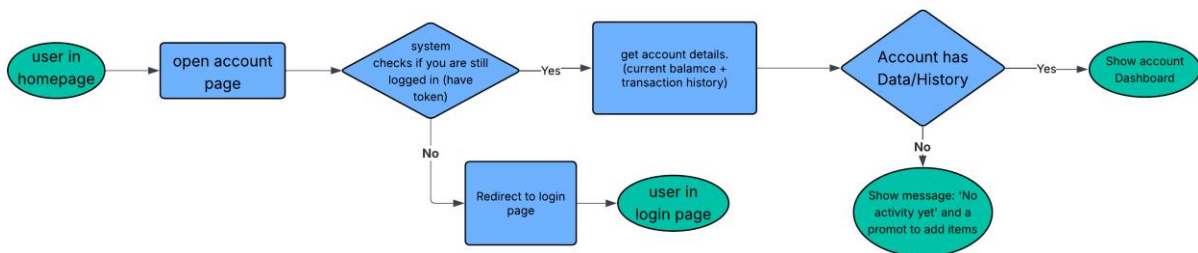


Figure 17 View account user flow diagram

- All flows in the system follow a unified communication pattern. Every client request passes through the API Gateway, which is responsible for JWT token verification (by communicating with the authentication server) before forwarding to any internal service. This ensures that authentication is handled at a single entry point, keeping internal services focused on business logic only. For operations that involve writes on multiple database nodes, a Two-Phase Commit (2PC) protocol is used to guarantee atomicity across distributed data.

7. Time Plan

Week 12

Creating servers, clients, and protocols for different functions of the system using Java Sockets.

Week 13

Converting the database ERD into schema, creating the database, making it distributed, and finally connecting it to the servers.

Week 14

Create the UI and connect it to logic.

Week 15

The final documentation

8. Selected technologies

Language: Java

Communication: Java Sockets (TCP)

Database: mariadb

DB Access: JDBC

Concurrency: Threads

9. Resources Needed

- **Software Resources:** Java Development Kit (JDK), Draw.io for Diagrams, IDEs (IntelliJ IDEA / Eclipse).
- **Hardware Resources:** Multiple laptops connected over a LAN/VPN to simulate the distributed environment.